APEX RETAIL STORE INTELLIGENCE PLATFORM

Edge Computer Vision & Real-Time Analytics Platform

Presented by: Sanket Sunil Nagathan

Role: Principal Software Architect

System Stack: PyTorch (YOLOv8) | FastAPI | ByteTrack | SQLite | Docker

SLIDE 2: PROBLEM STATEMENT & SOLUTION VISION

- **Retail Analytics Gap:** Physical stores are analytical black boxes compared to online stores. Footfall, dwell time, and department engagement are unmeasured.

- **Visitor Count Inflation:** Double-counting errors caused by overlapping camera views, returning visitors, and groups entering together.

- **Staff Activity Noise:** Employees moving around entrances/sales zones distort customer count metrics and checkout conversion statistics.

- **POS Integration Silo:** Sales transactions log checkout data but lack details about the physical customer journey and store behavior.

- **Solution Vision:** Integrates local edge cameras with sales checkouts to construct a unified, staff-filtered customer conversion funnel.

SLIDE 3: END-TO-END SYSTEM ARCHITECTURE

- **CCTV Feeds:** Feeds raw video streams (Store 1: Single Entrance/Exit, Store 2: Dual Entry/Exit + Sales Floor + Billing Cams).

- **Edge CV Layer (detect.py):** Runs local YOLOv8 human detection and ByteTrack coordinate tracking.

- **FastAPI Gate (main.py):** Ingests structured JSON events and validates schemas with logging middleware.

- **SQLite Persistence (store.db):** Normalized local store for events, transaction details, and processed video clip states.

- **Analytics Engine (sessions.py / metrics.py):** Reconstructs customer sessions, links cross-camera tracks, and matches sales.

- **Web Dashboard UI (dashboard.html):** Modern widescreen UI displaying real-time KPIs, conversion funnels, and heatmaps.

SLIDE 4: SOFTWARE ARCHITECTURE & DESIGN PATTERNS

- **Layered Architecture:** strict decoupling of ingestion, database, metrics engine, and presentation layers.

- **Repository Pattern (storage.py):** Encapsulates SQLite queries inside a modular Store class, isolating raw SQL queries.

- **Strategy Pattern (detect.py):** Uniformly selects uniform detection rules ('dark' vs 'pink_black' shirts) based on store layout profiles.

- **Factory Pattern (emit.py):** Normalizes raw coordinates and metadata to standard event payloads via make_event.

- **Data Transfer Objects (DTOs):** Uses Pydantic models (IngestRequest) to validate input structures at API ingress.

- **Dependency Inversion:** Decouples DB path injections, allowing easy switching between production, testing, and memory databases.

SLIDE 5: COMPUTER VISION & TRACKING ARCHITECTURE

- **Low-Latency Inference:** YOLOv8 Nano model (yolov8n.pt) detects human targets with automatic GPU/CPU fallbacks.

- **ByteTrack Association:** Centroid-based tracking maintains visitor IDs across frames and minor occlusions.

- **Lightweight Re-ID Heuristics:** Centroid proximity combined with a mean RGB appearance histogram on torso crops.

- **Why Heuristics:** Avoids the heavy GPU memory usage and CPU latency of deep learning Re-ID models (like OSNet).

- **Exited Gallery Sweeper:** Sweeps inactive tracks after 5 seconds to allow 10-minute reentry matching.

- **Layout-Driven Zones:** Maps coordinates strictly to zones defined in store_layout.json to prevent coordinate bleeding.

SLIDE 6: EVENT-DRIVEN DATA MODEL

- **Transactional Event Log:** Centerpiece database table tracking ENTRY, EXIT, ZONE_ENTER/EXIT, and BILLING events.

- **Idempotent Ingestion:** Deterministic event UUIDs (uuid.uuid5) discard duplicate database entries on restarts.

- **Session Reconstruction:** Groups visitor ID suffixes (#n re-entries) into a single physical session (Session class).

- **Cross-Camera Linking:** Links floor and checkout tracks (separate IDs) to entry sessions using a 60s correlation window.

- **Data Flow Sequence:** Frame Detection -> Event Generation -> API POST Ingestion -> SQLite Ingest -> Session Build -> Analytics.

SLIDE 7: ANALYTICS & INTELLIGENCE ENGINE

- **Funnel Stages:** Tracks conversion rates: Entry (100%) -> Zone Visit -> Queue Join -> Purchase checkout.

- **Dwell Time Heuristics:** Uses maximum zone dwell checkpoint, preventing tracking gaps from inflating stats.

- **Queue Abandonment:** Calculates queue joins against checkouts to isolate bottleneck delays.

- **POS Match Engine:** Time-matches checkouts to active billing sessions within a 5-minute pre-checkout window.

- **Anomaly Detection:** Flags transactions logged outside store hours or checkouts missing corresponding entrance events.

SLIDE 8: SCALABILITY & RELIABILITY

- **Fast Hot-Reload Startup:** Instantly reloads event logs from host JSONL backup files, skipping slow video processing on start.

- **Structured Logging:** Starlette middleware logs latency, trace IDs, and HTTP status codes to stdout.

- **Database Scaling:** Repository design enables switching from SQLite to PostgreSQL without code rewrites.

- **Test Suite Coverage:** Unit and integration tests verify trackers, metrics, and API endpoints.

SLIDE 9: CHALLENGES SOLVED & KEY ACHIEVEMENTS

- **Zero Re-entry Inflation:** Deduplicates returning visitors within a 10-minute threshold.
- **Dual Entry Merging:** Combines parallel entry feeds (Store 2) into a single visitor session.
- **Durable Staff Filtering:** Excludes employees via color classification over 3 consecutive frames.
- **Ingest Collision Guards:** Hashed event IDs prevent live-watcher and seeder data collisions.

SLIDE 10: FUTURE ROADMAP & TECHNICAL Q&A

- **Scale Ingestion:** Move to PostgreSQL for centralized, multi-store cloud databases.

- **Stream Orchestration:** Introduce Apache Kafka to route events from camera arrays.

- **Advanced Re-ID:** Upgrade to local Vision-Language Models (VLMs) for uniform classification.

- **Demo Highlights:** Show staff filtering, checkout funnels, and store conversion rates.

- **Sample Q&A:** Prepare for questions on lighting changes (handled by torso normalization) and network outages (handled by edge JSONL buffering).